

Bank Account Management System using Java OOP Concepts

1) Objective / Aim

To design and implement a Bank Account Management System in Java using object-oriented programming concepts. The program demonstrates classes, objects, inheritance, method overriding, constructors, deposits, withdrawals, interest calculation, account display, and fund transfer operations.

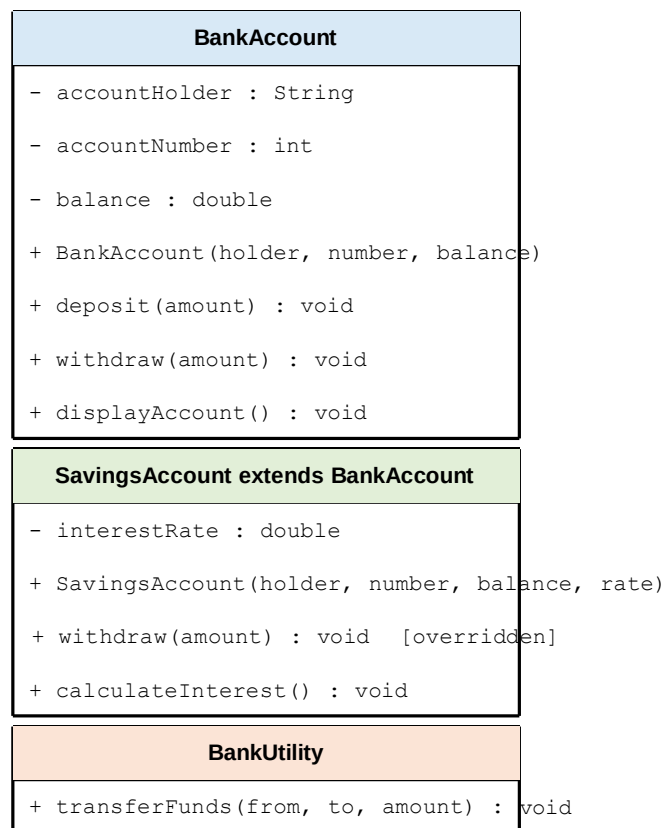
2) Execution Steps

1. Open a Java IDE or a command prompt/terminal.
2. Create a Java source file named **BankAcc.java**.
3. Enter the given program containing the **BankAccount**, **SavingsAccount**, **BankUtility**, and **BankAcc** classes.
4. Compile the program using **javac BankAcc.java**.
5. Run the program using **java BankAcc**.
6. The program first displays the details of Alice's and Bob's accounts.
7. It performs deposit and withdrawal operations on both accounts.
8. Interest is calculated for the savings account.
9. Rs. 500 is transferred from Alice's account to Bob's account.
10. Finally, the updated account details are displayed.

3) Algorithm

1. Start the program.
2. Create a base class **BankAccount** with account holder, account number, and balance.
3. Define a constructor to initialize the account details.
4. Define **deposit()** to add the given amount to the balance.
5. Define **withdraw()** to withdraw money when sufficient funds are available.
6. Define **displayAccount()** to print account details.
7. Create **SavingsAccount** by extending **BankAccount**.
8. Add an interest rate to the savings account and override **withdraw()** so that a minimum balance of Rs. 500 is maintained.
9. Define **calculateInterest()** to calculate interest and add it to the savings balance.
10. Create **BankUtility** with a static **transferFunds()** method for transferring money between accounts.
11. In the **main()** method, create Alice's normal account and Bob's savings account.
12. Display both accounts, perform deposits and withdrawals, calculate interest, and transfer Rs. 500 from Alice to Bob.
13. Display the final balances.
14. Stop the program.

4) UML Class Diagram



Relationship: SavingsAccount inherits from BankAccount. BankUtility provides a static operation for transferring funds between BankAccount objects.

5) Source Code

The following is the source code provided for the experiment. It is included as selectable text.

```
// Base class: BankAccount
class BankAccount {
    protected String accountHolder;
    protected int accountNumber;
    protected double balance;

    // Constructor
    public BankAccount(String accountHolder, int accountNumber, double balance) {
        this.accountHolder = accountHolder;
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    // Deposit method
    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount + " | New Balance: " + balance);
    }

    // Withdraw method
    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount + " | New Balance: " + balance);
        } else {
            System.out.println("Insufficient funds!");
        }
    }

    // Display account details
    public void displayAccount() {
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }
}

// Derived class: SavingsAccount (inherits BankAccount)
class SavingsAccount extends BankAccount {
    private double interestRate;

    // Constructor
    public SavingsAccount(String accountHolder, int accountNumber, double balance, double interestRate) {
        super(accountHolder, accountNumber, balance);
        this.interestRate = interestRate;
    }

    // Method overriding: withdraw with restriction
    @Override
    public void withdraw(double amount) {
        if (balance - amount < 500) { // Minimum balance restriction
            System.out.println("Cannot withdraw. Minimum balance of 500 must be maintained.");
        } else {
            super.withdraw(amount);
        }
    }

    // Interest calculation
    public void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance += interest;
        System.out.println("Interest added: " + interest + " | New Balance: " + balance);
    }
}

// Utility class for fund transfer
class BankUtility {
    public static void transferFunds(BankAccount from, BankAccount to, double amount) {
        if (amount <= from.balance) {
            from.withdraw(amount);
            to.deposit(amount);
            System.out.println("Transferred " + amount + " from " + from.accountHolder + " to " + to.accountHolder);
        } else {
            System.out.println("Transfer failed. Insufficient funds in source account.");
        }
    }
}

// Main class
public class BankAcc {
    public static void main(String[] args) {
        // Create accounts
        BankAccount acc1 = new BankAccount("Alice", 101, 2000);
        SavingsAccount acc2 = new SavingsAccount("Bob", 102, 3000, 5);

        // Display accounts
        acc1.displayAccount();
        acc2.displayAccount();
    }
}
```

```
    // Perform operations
    acc1.deposit(500);
    acc1.withdraw(1000);

    acc2.deposit(1000);
    acc2.withdraw(2700); // Will check minimum balance
    acc2.calculateInterest();

    // Fund transfer
    BankUtility.transferFunds(acc1, acc2, 500);

    // Final account details
    acc1.displayAccount();
    acc2.displayAccount();
}
}
```

6) Output

The following output is the terminal-style output supplied for this program.

```
Account Holder: Alice
Account Number: 101
Balance: 2000.0
Account Holder: Bob
Account Number: 102
Balance: 3000.0
Deposited: 500.0 | New Balance: 2500.0
Withdrawn: 1000.0 | New Balance: 1500.0
Deposited: 1000.0 | New Balance: 4000.0
Withdrawn: 2700.0 | New Balance: 1300.0
Interest added: 65.0 | New Balance: 1365.0
Withdrawn: 500.0 | New Balance: 1000.0
Deposited: 500.0 | New Balance: 1865.0
Transferred 500.0 from Alice to Bob
Account Holder: Alice
Account Number: 101
Balance: 1000.0
Account Holder: Bob
Account Number: 102
Balance: 1865.0
```

The output shows the initial account details, deposits, withdrawals, interest addition, fund transfer, and final account balances.

7) Result

The Bank Account Management System was successfully implemented in Java. The program demonstrates object creation, inheritance, method overriding, deposit and withdrawal operations, interest calculation, account display, and transfer of funds between accounts.

Key OOP Concepts Demonstrated

Concept	Where it is demonstrated
Class and Object	BankAccount, SavingsAccount, and objects acc1 and acc2
Inheritance	SavingsAccount extends BankAccount
Method Overriding	SavingsAccount overrides withdraw()
Constructor	BankAccount and SavingsAccount constructors
Encapsulation	Account fields and class methods
Static Method	BankUtility.transferFunds()